SIMATS ENGINEERING
ASSESSMENT TOOL 2:

Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1519

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I Jannathul Firdaus. N (192573025) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Case Study

1. **Case Study 1: Smart Retail Inventory Management System**

   A retail chain implements a cloud-based inventory system connected to store devices.Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. **Case Study 2: Cloud-Based Document Collaboration System**

   An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

   Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. **Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.

Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Cloud Based Document collaboration System:

Introduction:

A cloud based document collaboration System similar to google docs allow multiple user to create edit and share documents through web browsers. It uses distributed cloud storage, web API and version control to support real time collaboration

Architecture and working:

Client Layer: Web browsers and mobile devices provide access to documents.

Web API layer: API transmit editing operations and synchronize updates between clients.

Application Layer: Cloud Server process document requests and coordinate collaborative editing.

Cloud Storage: Distributed Storage maintains document and their versions.

Infrastructure: Distributed Servers help reduce latency and improve global availability.

Security layer: Encryption, identity management and access permission.

Working:

User sign in and open a document through a browser. Editing operation are sent to cloud Services through API. The system processes the changes and synchronizes them with other authorized users. updated documents are saved in cloud storage. while version control maintain document history.

Advantages:

* Multiple users can edit document simultaneously
* Documents available from different devices and locations.
* Previous version can be reviewed or restored.
* Supports increased number of users and documents

Challenges and Solutions:

* unauthorized access: Implement identity management and access permissions.
* Data interception: Use HTTPS and encryption at rest.
* Data Loss: Maintain backups and document version history.
* Network latency: Use geographically distributed infrastructure and efficient synchronization.

Smart Retail Inventory Management System

Introduction:

A Smart Retail Inventory Management System uses cloud computing to monitor stock levels, record sales, and forecast product demand across multiple retail stores. It integrates POS system, web APIs, cloud storage and analytics services.

Architecture and Modeling:

Client layer: POS systems and dashboards allow employees to record sales and monitor inventory

API layer: API transfer sales transactions and inventory updates between store device and cloud services.

Cloud storage: Stores product details, transaction logs and analytics datasets.

Platform service: Analytics and ML services analyze historical sales data and forecast future demand.

Security layer: VPN, firewall protect communication and unauthorized access.

Working:

When a product is sold, the POS system sends transaction details through an API to cloud. The backend processes the transaction and update

Inventory records. The updated stock information is displayed on dashboards. Analytics services examine historical sales data to predict demand and support stock replenishment.

Advantages:

* Maintains updated stock information across stores.
* Cloud resources can expand according to transaction volume.
* Reduces the need for extensive on premises infrastructure.
* Helps prevent Overbooking and stock shortages.
* Enables consistent inventory monitoring across branches.

challenges and solution:

Security threats: Use VPN, firewall, API gateways and strong authentication.

Network failure: use reliable connectivity and temporary local transaction storage.

High transaction volume: Apply auto-scaling and load balancing.

Inaccurate stock records: Use transaction validation and data synchronization.

Online food ordering platform:

Introduction:

An online food ordering platform is a cloud based application that allows customers to browse menus place food orders and track order status using web browsers or mobile applications. It integrates RESTful APIs, cloud storage virtual machines and managed Kubernetes

Architecture and working

Frontend Layer: web browsers and mobile application allow customers to browse menus and place orders.

RESTful Web API: Enable communication between frontend clients and backend services using HTTP such as GET POST.

Backend Service: Process customer requests validate order calculate totals and manage order information.

virtual machine: provide computing resource to run application components.

cloud Storage: Stores menu images, invoice and customer-related files.

Security layer: Authentication tokens, HTTPS and role-based access control protect application resources.

Working:

Customer access the platform through a website or mobile application. The frontend requests menu details through REST API. When a customer places an order the backend validates the request records the order and returns confirmation. Relevant data is stored in cloud databases and storage services. Kubernetes scales application services during peak demand.

Advantages:

* Handles increases in customer traffic using cloud resources and Kubernetes.
* Redundant services and recovery mechanism help reduce downtime.
* Customer can order food anytime through web or mobile clients.
* API connect frontend applications with backend services.
* HTTPS and authentication protect communication and access.
* Resources can be adjusted according to demand.